Research Document

Project Title: Gamified App For Student

Author: Shuai Jiang

Date: 12/10/2025

1. Introduction

1.1 Problem Statement

Higher education students today face a unique set of challenges. They often struggle with disorganized tasks from various sources, difficulty in maintaining motivation for self-directed study, and a sense of social isolation on a large campus. Balancing these academic pressures with personal well-being is a challenge.

1.2 Project Vision & Goals

This project is a mobile app designed to help students manage their studies and personal growth in a more engaging way. To keep students motivated, the app uses several game-like features. By completing personal goals and real-world activities, students can earn Points, unlock achievement Badges, and compete on staged Leaderboards. What makes this app different is its focus on the real world. Instead of encouraging more online chatting, its key goal is to provide students with a tool to practice and improve their face-to-face social skills.

2. Gamification

2.1 How It Works

Gamification is the process of integrating game design elements and principles into non-game contexts. The goal is to increase user engagement and motivation through the use of game elements such as points, badges, leaderboards, and more(Hamari, 2019)(Deterding et al., 2011)(Robson et al., 2015). For my project, this is the core strategy to solve the problem of student motivation decline.

The system works by creating a positive feedback loop. When a student completes a real-world task that is beneficial to them, the app provides an immediate virtual reward. This psychological reward helps reinforce the positive behavior, making the student more likely to repeat it. The key elements I plan to use are:

Points: Points in gamification are not just random numbers; they are a powerful tool to engage users, motivate behavior, and provide instant feedback. Whether it's improving sales, enhancing learning, or boosting app engagement, points can dramatically increase participation and satisfaction(Jacob, 2024). This is the most direct form of feedback and the core currency of the app's economy. Completing almost any positive action, such as finishing a task or a successful focus session, will grant the user experience points (XP). These points serve a dual purpose: they act as a clear measure of the user's effort, contributing to their level progression, and they also function as "credits" that can be spent in the in-app shop, giving every point earned a tangible value.

Badges: In the gaming world, virtual rewards — like badges — recognise players' growing skills and heroic feats. This makes them powerful engagement tools.

Outside of gaming, they act as simultaneous goal-setting and reward devices. They give us something to aim for and encourage us to pursue objectives, and explore and overcome obstacles.

In summary, gamification badges exist for the following purposes:

- Motivating action and participation.
- Motivating exploration and collaboration.
- Marking progress or status.
- Acting as reward or recognition.
- Enabling goal and objective setting(Harry, 2022).

To add more depth, I plan to implement a tiered badge system (e.g., bronze, silver, gold). For example, a student might earn a bronze "Focus Master" badge after 10 hours of focused study, a silver one after 50 hours, and a gold one after 150 hours. This tiered structure provides a continuous sense of progression and encourages long-term commitment.

Leaderboards: A leaderboard is a high score list. Leaderboards rank players

according to their relative success, measuring them against a certain criterion. As a result, leaderboards can be used to identify the best performers of a certain activity(Cloke, 2021). To add a social and competitive element, the app will use a staged or league-based leaderboard system. Instead of a single, global ranking where new users might feel discouraged by the huge gap with top players, this system places users into small, competitive groups (e.g., 30-50 people) with others at a similar activity level. The competition runs on a weekly cycle. At the end of the week, the top performers in each group are promoted to a higher league, while the lowest performers may be demoted. This structure ensures that every user, regardless of their overall rank, has an achievable and relevant goal each week—either fighting for promotion or striving to avoid demotion—which keeps the competition consistently fair and engaging.

2.2 How To Implement

To ensure the rules are consistent and secure, the core logic for the gamification engine will be implemented on the backend (server-side) using Python and Django.

When the user completes an action in the mobile app (the frontend), the app will send a request to my backend API (e.g., POST /api/tasks/complete).

The backend will then verify this action, calculate the appropriate points and check if any badges have been earned.

Finally, the backend will update the user's data in the database and send a confirmation back to the mobile app, which will then display a notification to the user (e.g., "You earned 20 points!").

By handling this logic on the server, I can easily adjust the rules (like how many points a task is worth) without needing to update the mobile app itself, and it ensures a fair and consistent experience for all users.

2.3 Potential Challenges and Issues

While gamification is a powerful tool, my research indicates several potential issues that I must carefully manage during development:

- **The Overjustification Effect:** This is a key risk. By providing external rewards (Points, Badges) for activities that should be intrinsically motivating (like learning or socializing), there's a danger that users might start to feel they are *only* doing these activities for the points. If the rewards are removed, their intrinsic motivation might be lower than when they started.

  Strategy: I will keep the rewards supportive rather than transactional. The UI/UX, inspired by Headspace , will focus on "growth" and "achievement" rather than just "earning," to protect that intrinsic motivation.

- **Unbalanced Economy:** It will be a challenge to balance the points earned from different activities (e.g., finishing a task vs. a 1-hour focus session vs. a social interaction). If one activity is significantly more "profitable" than others,

users might focus only on that single activity, which defeats the app's purpose of promoting a balanced student life.

Strategy: The backend-driven rules system is crucial. It will allow me to easily tune and adjust the point values for each activity during testing, to ensure all core behaviors are encouraged relatively equally.

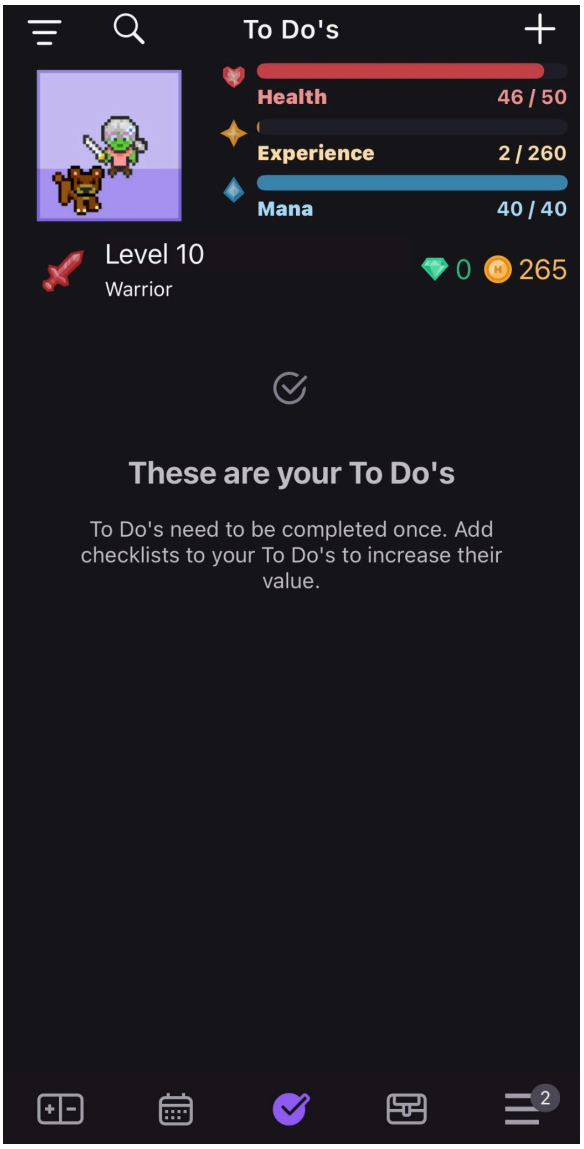
- **Leaderboard Demotivation:** While the Leaderboard is an **Essential** feature, competition can still be stressful for some students. Users who consistently find themselves at the bottom of their league might feel demotivated and abandon the app.

Strategy: My primary strategy to combat this is the **design of the leaderboard itself**.

1. By implementing the **staged-league system** (inspired by Duolingo), users are not competing against everyone, but only within a small group of similarly active peers. This makes the competition feel fair and achievable .
2. Secondly, while the leaderboard provides *competitive* motivation, the app will equally emphasize **personal progression** through **Badges** and **Levels** . This gives users a parallel path to feel a sense of achievement, even if they aren't at the top of their league that week.
3. Finally, by adapting concepts like the "Streak Freeze" from Duolingo for our "Credits" shop , we can help reduce the *anxiety* associated with performance, which indirectly affects the leaderboard experience.

3. Market and Competitive Analysis

3.1 Analysis: Habitica(Habitica, 2020)



3.1.1 Core Concept

After using it for a week, I found that Habitica is an innovative productivity tool. My understanding is that it changes a user's real-life tasks (including habits, daily to-do, and normal tasks) into a complete role-playing game (RPG). I saw that users get experience and gold by completing tasks in real life, which upgrades their virtual character in the game. I believe this method makes the process of completing boring tasks full of fun.

3.1.2 Gamification Mechanics

Some important features I identified are: Experience Points (XP) & Levels; Gold & Reward Shop; Health Points (Health) & Punishment; Character Role-Playing

Elements: Equipment, Pets, and Mounts; and a Social System where users can form "Guilds".

3.1.3 Something Can Learn From This App

A. Valuable Concepts I Can Borrow

Punishment Mechanism: I think Habitica's "lose Health" punishment is very effective because it makes the user's promised tasks feel more important. For my own app, I can learn from this idea and use a more gentle consequence method. For example, if a user fails their "Focus Mode" challenge, they might lose a small number of points. I feel this is stronger than just getting "no reward."

Internal Economy System: I also found that Habitica's Gold and Shop system gives a second value to the user's effort, besides just upgrading. I believe I can learn from this point. In my app, I can let students use their earned points not only to level up, but also to exchange for personalized rewards (like new app themes or avatar looks) in a simple "shop". In my opinion, this will make earning points have more purpose.

B. Elements I Should Avoid

Overly Complex RPG System: For me, this is the most important point. I found that Habitica’s classes, equipment, and pet systems are very attractive to game players, but I think they might distract students' attention. My opinion is that users may spend a lot of their study time to research these game systems, which is opposite to my app's main goal of "improving study efficiency." For my project, I want to keep it simple, to make sure the gamification is for motivation, not the final goal itself.

Unique UI Style: I can accept Habitica's pixel art style, but I agree it is not for everyone. For an app that I want to build to help students focus and grow, I believe a more modern, simple, and calm interface is a better strategic choice.

C. Concepts I Can Adapt

Changing "Guilds" into Offline "Study Groups": I saw that Habitica's "Guilds" is a pure online social function. I think I can learn from its "team work" idea and adapt it to serve my "offline social" core goal. For example, in my app, users could form "Study Groups." When group members use the Bluetooth function to confirm they had an offline meeting, the whole group would get extra rewards. I think this will effectively motivate real study communication.

3.2 Analysis: Forest(Forest, 2016)

3.2.1 Core Concept

After using Forest, I found its core concept is to use a simple and beautiful metaphor—growing a tree—to help users stay focused and put down their phones. It's essentially a gamified timer that gives users a positive visual reward for their period of concentration.

3.2.2 Gamification Mechanics

Forest's gamification is very direct and effective. The main mechanics I identified are:

Positive Reward (Positive Reinforcement): Successfully completing a focus session results in a new tree being added to your personal forest.

Negative Consequence (Punishment): If you leave the app during a focus session, the growing tree will wither and die. This dead tree permanently stays in your forest as a reminder of the failure.

Accumulation & Visualization: Over time, successful sessions build a dense, personalized forest, visually representing all the time the user has dedicated to focusing.

In-App Currency: Successful sessions also earn coins, which can be used to unlock new tree species.

3.2.3 Something Can Learn From This App

A. Valuable Concepts I Can Borrow

The Power of Consequences: The idea of a tree withering is a very powerful psychological penalty. It's much more effective than simply not getting a reward. This reinforces my decision to include a penalty for failing the "Focus Mode" in my app, such as losing a small number of points.

Visualizing Effort: The concept of building a forest is a great way to visualize accumulated effort. I can adapt this idea not by building a forest, but perhaps by creating a "virtual trophy room" or a customizable profile page where unlocked badges are displayed, creating a visual record of achievement.

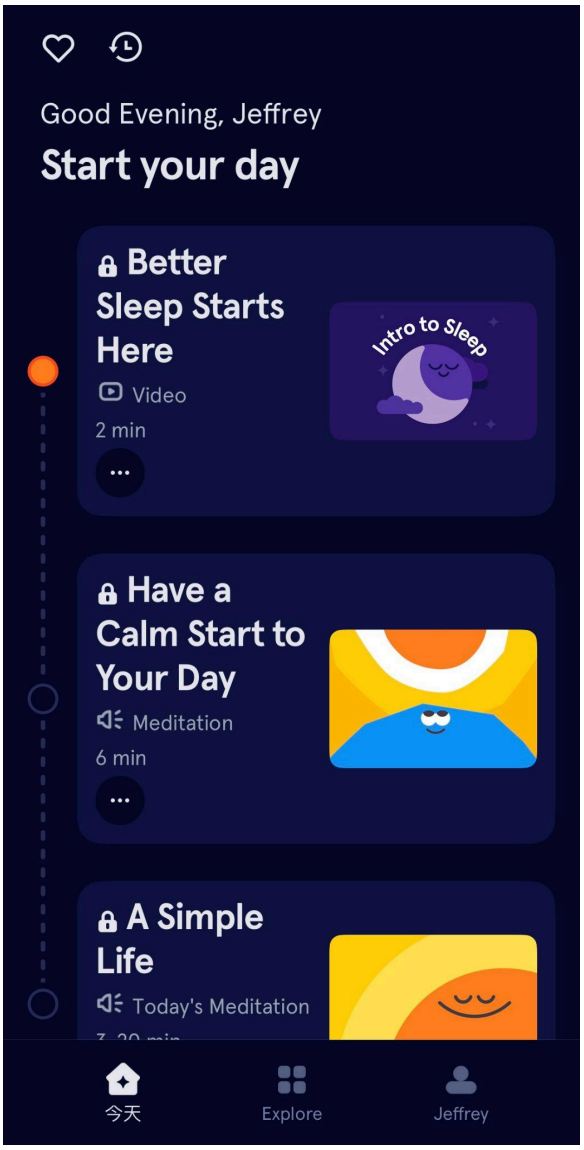
B. Elements I Should Avoid

Single-Purpose Focus: Forest is an excellent app, but it does one thing: focus timing. My project is a multi-functional companion for students. Therefore, I cannot make the entire user experience revolve around a single feature. The Focus Mode needs to be a well-integrated part of a larger ecosystem of features.

C. Concepts I Can Adapt

Adapting "Group Focus": Forest allows multiple users to plant a tree together online, and if one person gives up, everyone's tree dies. I can adapt this concept for my "offline social" goal. For example, when members of a "Study Group" in my app confirm they are meeting in person via Bluetooth, they could start a "Group Focus Session." If the group successfully completes the session, everyone receives a significant point bonus, which would strongly encourage productive, real-world study groups.

3.3 Analysis: Headspace(Headspace, 2024)



3.3.1 Core Concept

After trying Headspace, my analysis is that it is a subscription-based service focused on mental well-being. It offers a large library of guided meditations, sleep aids, and mindfulness exercises. I see that it positions itself as a "gym membership for the mind," aiming to make mindfulness an accessible daily habit.

3.3.2 Gamification Mechanics

I found that Headspace's use of gamification is very subtle and supportive, rather than competitive. The main mechanics are:

Daily Streaks: It tracks and rewards the number of consecutive days a user completes a meditation session.

Progress Tracking: It visually displays the user's total accumulated meditation time.

Positive Reinforcement: The entire app is filled with encouraging language and calming, rewarding animations after a session is completed.

3.3.3 Something Can Learn From This App

A. Valuable Concepts I Can Borrow

Intuitive and Calming UI/UX: For me, the biggest lesson from Headspace is the importance of a clean, calm, and positive user interface. The app uses soft colors, friendly illustrations, and a simple layout that feels supportive, not stressful. As you pointed out, users don't need to specifically learn how to use it. This principle of a **"low learning curve"** is something I must apply to my own design, to ensure my app helps reduce student stress, not add to it.

Structured Daily Engagement: Headspace offers a "Meditation of the Day" which encourages users to open the app daily. I can borrow this idea for my project by creating a "Social Challenge of the Week" or a "Daily Wellness Tip" to provide a simple, recurring reason for users to check in.

B. Elements I Should Avoid

Subscription Model: Headspace's main business is selling access to content. My app, however, is a tool, not a content library. I will avoid placing any of my core productivity or social features behind a paywall.

Passive Experience: I noticed the primary user action in Headspace is passive (listening). My app's goal is to encourage active behaviors like completing tasks and meeting people. Therefore, I need to ensure my features are interactive and prompt the user to do something.

C. Concepts I Can Adapt

Adapting "Mindful Moment" Notifications: Headspace can send gentle notifications to remind users to take a break. I can adapt this into a much simpler, optional **"Wellness Check-in"** feature. This could be a single daily notification asking, "How are you feeling today?" which users can quickly respond to for a small point reward. I think this would be a lightweight and effective way to address the "Mental Health" aspect of my project.

3.4 Analysis: Duolingo(Duolingo, 2025)

3.4.1 Core Concept

My analysis of Duolingo is that it is a language-learning platform that has successfully turned the entire, often difficult, process of learning a language into a fun and accessible game. I found that it breaks down complex language skills into small, bite-sized lessons and uses a powerful gamification engine to motivate users to practice every day.

3.4.2 Gamification Mechanics

Duolingo is a masterclass in gamification. The key mechanics I have focused on are:

Leagues (Leaderboards): The core competitive feature, placing users in weekly competitions against a small group of peers.

Streaks: Tracks the number of consecutive days a user has met their daily goal.

Experience Points (XP): Earned for completing lessons, and serves as the basis for the leaderboard ranking.

In-App Currency (Gems/Lingots): A virtual currency earned through use, which can be spent in the shop.

Achievements (Badges): A set of badges awarded for hitting various milestones (e.g., total XP earned, number of perfect lessons).

3.4.3 Something Can Learn From This App

A. Valuable Concepts I Can Borrow

The League System Model: Duolingo's league-based leaderboard is the **direct model** for my app's staged leaderboard. The way it places users in small, weekly competitive groups (e.g., 30 people) with clear promotion and demotion zones is something I plan to implement directly. I believe this system is brilliant because it makes the competition feel fair and consistently motivating, which will be very effective for students.

The Streak & In-App Economy: The "Daily Streak" is a proven mechanic for building habits, which I will definitely include in my app. More importantly, Duolingo's **"Streak Freeze"**—an item that can be bought with Gems to save a streak—is a perfect example for my in-app "Credits" store. It shows how to offer items that provide real value and reduce user anxiety, rather than just being cosmetic. This directly relates to my supervisor's suggestion to include a way for users to "buy things".

B. Elements I Should Avoid

Aggressive Notifications: I know from experience that Duolingo is famous for its very persistent notifications. For my app, which is aimed at helping students reduce stress, I will adopt a calmer and more respectful notification strategy. The notifications should be helpful reminders, not a source of pressure.

Linear Learning Path: Duolingo's lesson structure is very linear and prescribed; you must complete one level to unlock the next. My app is not a course, but a toolbox. Therefore, the user experience must be flexible, allowing students to use the features they need (like the To-Do List or Focus Mode) in any order they choose.

C. Concepts I Can Adapt

The main adaptation from Duolingo is taking its core motivational engine (Leagues and Streaks), which is designed for a single, linear goal (learning a language), and applying it to the more diverse and user-driven context of student life. The challenge will be to balance the point rewards from different activities (studying, socializing, etc.) to create a fair and engaging overall experience.

3.5 Comparative Analysis of Other Apps

This table summarizes the key insights learned from other relevant applications.

App Name	Relevant Feature in Our Project	Key Gamification Method	Strengths to Borrow	Weaknesses / Differences
Habitica	**Gamification Engine & Personal Task Management**	A deep RPG system with XP, Health points (punishment), Gold (in-app currency), Equipment, mounts, pets, and social Guilds.	The **"Consequence Mechanism"** (losing health) is powerful. The **"In-App Economy"** (spending currency) gives points real value.	The **RPG system** is **overly complex** and can be a distraction. The social features are online-only. The pixel art style is niche.
Forest	**Focus Mode**	Successfully focusing grows a tree; failing kills the tree. The accumulated trees form a visual forest.	The **"group focus"** concept is a perfect inspiration for our offline **"Study Groups"**.	**Forest is almost entirely a focus timer.** However, the application we are developing is a multifunctional student assistant app.
Headspace	**UI/UX Design & Mental Well-being Module**	It primarily uses daily streaks and a calm, encouraging UI/UX to build a habit.	**The simple, calm and intuitive interface** is deeply loved by users, and it also has a **simple operation logic**.	Headspace is about **passive content consumption (listening).** Our app's core is to encourage **active user behaviors (completing tasks, socializing).**
Duolingo	**Gamification Engine (Leaderboards & Economy)**	Its core loop is driven by a highly competitive weekly league system, daily streaks, and virtual currency (gems) used to purchase items such as "Streak Freeze".	**The league-based leaderboard** is the perfect model for our staged leaderboard system. The "Streak Freeze" is an excellent idea for our in-app "Credits" store.	The **learning path** is **highly structured**, which may make people **feel monotonous and repetitive, and a bit boring.** Our application needs to become a flexible toolbox. Its notification strategy might be overly aggressive.

3.6 Project's Position

After analyzing these successful applications, we can find that they are usually designed as universal tools for a broad audience. They lack specific attention to the lives of students.

"Gamified App For Students" precisely targets this gap. Its unique value is based on two core principles:

Specifically for students: The functions of this application are tailored to the daily schedules and challenges faced by students.

Encouraging offline social interaction: The key feature of this project lies in its innovative Bluetooth-based function, which rewards face-to-face interactions.

4. Technology Investigation

4.1 Mobile Platform Investigation

4.1.1 Comparing Android vs. iOS Platforms

In choosing a development platform, I first compared the two main ecosystems: Android and iOS.

Android (Google):

- **Advantages:** This is an open ecosystem. Developers can freely test on any device, and the process for publishing to the Google Play Store is relatively faster and has fewer restrictions. It has a very high global market share, especially in Europe and Asia, which provides a broader potential user base for the application.
- **Disadvantages:** "Fragmentation" is the biggest challenge. Due to the large number of different manufacturers (Samsung, Google, Huawei, etc.), screen sizes, and operating system versions, ensuring the app runs well on all devices requires more testing and adaptation effort.

iOS (Apple):

- **Advantages:** This is a closed, highly unified ecosystem. Developers only need to develop for a few iPhone and iPad models, meaning the device fragmentation problem is almost non-existent. This makes it much easier to guarantee a high-quality and consistent user experience.
- **Disadvantages:** The development and publishing process is stricter. Development must be done on macOS (requiring an Apple computer), and apps are subject to a strict and lengthy review process by Apple when submitting to the App Store.

4.1.2 Comparing Development Stacks (Kotlin vs. Flutter)

After deciding on a mobile-first strategy, I compared two mainstream development technologies: Native (Kotlin) and Cross-Platform (Flutter).

Comparison Dimension	Native (Kotlin for Android)	Cross-Platform (Flutter with Dart)

Performance	Code is compiled directly to native machine code, fully leveraging device performance.	Flutter bypasses native components and draws UI directly on a canvas, with performance close to native.
Development Efficiency	Development services one platform at a time (Android).	A single codebase can be compiled for both Android and iOS apps, theoretically saving 50% of the time.
UI Consistency	The app's look and feel will perfectly match the system habits of Android users.	Because Flutter has its own rendering engine, the app's appearance and interaction are almost identical on Android and iOS.
Device Feature Access	Can seamlessly and reliably call all native APIs (like Bluetooth, GPS, camera).	Needs to use "channels" to call native APIs. For complex features like Bluetooth, it may require extra configuration and reliability is slightly lower than native.
Learning Curve	This is a personal advantage, as I have prior experience with Kotlin.	Requires learning a new language (Dart) and a completely new responsive UI framework (Flutter) simultaneously.

4.1.3 Decision: Native Android with Kotlin

In choosing a development platform for this gamified student app, I compared the two major ecosystems—Android and iOS—and evaluated them in the context of our project goals. The Android ecosystem is open and flexible, allowing developers to test across a wide range of devices and publish to the Google Play Store with fewer restrictions. Its global reach is a major advantage; as the referenced article states: “The Android market share of 71.42% in 2025 outnumbers the iPhone global market

share of 27.93%, with a total of 3.5 billion active Android users worldwide(Delgado, 2025).” This broad user base is particularly valuable for an application targeting students with diverse device access.

In contrast, iOS offers a more unified and consistent environment due to its limited number of device models, which simplifies UI/UX consistency and performance tuning. However, its stricter development constraints (macOS requirement, lengthy App Store review, and closed ecosystem) reduce development agility.

Based on this analysis, I have chosen Native Android development with Kotlin for the initial implementation. The project requirement states that the app may be built for “Android and/or iOS,” and focusing on a single platform will allow me to deliver a more complete, stable, and polished product within the project timeframe. While Flutter’s cross-platform capability is appealing, adopting a new language and framework (Dart/Flutter) would introduce unnecessary learning overhead and technical risk. My existing experience with Kotlin removes this barrier and allows me to begin feature implementation immediately.

Additionally, the project’s core innovative feature (UC-RW-03) depends heavily on Bluetooth functionality. Native Android development ensures the most direct and reliable access to the Bluetooth stack, minimizing risks in the most technically complex part of the application. Looking ahead, the project’s decoupled backend architecture ensures that creating an iOS client in the future will be highly feasible, as most of the core logic will already exist and only the UI/client layer will need to be implemented.

4.2 Architecture: Decoupled (Client-Server) vs. Monolithic

Here is a brief introduction and justification for the specific tools I chose, including a comparison against other viable alternatives. I chose a decoupled (client–server) architecture, where the backend and frontend are developed as separate projects. The alternative would be a monolithic architecture, in which the frontend and backend are tightly combined into a single codebase. As noted in existing literature, “Monolithic architecture is a software design methodology that combines all of an application's components into a single, inseparable unit(GeeksforGeeks, 2024).” While this structure can simplify deployment for basic websites, it introduces major drawbacks in flexibility and scalability. In fact, “because of its rigidity, it is difficult to scale and maintain, which makes it difficult to adjust to changing needs(GeeksforGeeks, 2024).” These limitations make the monolithic approach unsuitable for a project that must support native mobile applications and future multi-platform expansion.

By contrast, a decoupled architecture “refers to a software design approach where the frontend (presentation layer) and backend (data and logic layer) operate independently, communicating via APIs(Gupta, 2023).” This separation allows me to design a flexible REST API on the backend that can serve any client—whether it is

our Android app, a future iOS app, or a website—making it a significantly more modern, scalable, and future-proof choice.

Because the backend is isolated from client-specific implementation details, updates to the mobile front end do not require backend modifications, and backend logic can evolve without impacting the mobile interface. This independence ensures that we can introduce new platforms in the future without rewriting core logic. Considering that our project's key functionalities (including the Bluetooth-module communication) are implemented server-side, adopting a decoupled architecture guarantees that when an iOS version is developed later, the backend foundation will already be complete. From a risk-management perspective, this architecture improves scalability, maintainability, and accelerates the rollout of additional client applications.

4.3 Backend Framework: Django vs. Flask

For the backend, I will be using Python with the Django framework. Another widely used option in the Python ecosystem is Flask. Flask is a lightweight and highly flexible micro-framework, giving developers significant freedom in structuring their applications. However, as noted in the literature, "Django is a high-level, batteries-included framework, while Flask is a lightweight, micro-framework(pythontutorials, 2025)." This means that Flask requires developers to manually integrate many essential components—such as authentication, database tools, and administrative interfaces—by selecting and configuring external libraries.

For my project, Django's "batteries-included" philosophy is a major advantage. Its built-in authentication system, ORM, and automatically generated admin interface provide robust out-of-the-box functionality that would require substantial manual work if implemented in Flask. Choosing Django allows me to save a significant amount of development time on these standard components and instead focus on the core, distinctive aspects of my application—particularly its gamification logic and interactive features.

4.4 Frontend Platform: Native Android (Kotlin) vs. Cross-Platform (Flutter)

For the mobile application, I will be building a native Android app using Kotlin. I considered using a cross-platform framework like Flutter, which could build for both Android and iOS from a single codebase.

However, the cross-platform approach would require me to learn a new language (Dart) and a completely new framework. By choosing native Android with Kotlin, I can focus on mastering one ecosystem. This allows for the best possible performance, direct access to all native device features (which may be important for the Bluetooth module), and a more polished user experience. Since the project requirement allows for "Android and/or iOS", I believe delivering one high-quality native application is a better strategy than potentially struggling with two platforms at once.

4.5 Production Database: PostgreSQL vs. MySQL/NoSQL

4.5.1 Two-Stage Database Strategy

For this project, I use a two-stage database strategy that matches the different phases of development of my gamified student app.

Research Document

- Stage 1 – Development: I use SQLite, because it is Django's default database, requires no separate server, and stores all data in a single file. This lets me quickly implement and test features such as task management, focus sessions, and basic gamification logic on my local machine.
- Stage 2 – Production: For the production-ready version, I plan to migrate to PostgreSQL so that my app can safely handle many students using the system at the same time, especially when they complete tasks, finish focus sessions, or confirm social interactions via Bluetooth.

SQLite is file-based and uses database-level locks; it "supports an unlimited number of simultaneous readers, but it will only allow one writer at any instant in time(anand, 2010)." This is acceptable for early development on my laptop, but not for a real multi-user deployment where many concurrent write operations are expected. In such a situation, PostgreSQL is more appropriate because it is a full client–server relational database designed for high concurrency and ACID transactions.

4.5.2 SQL vs NoSQL in the Context of My App

My backend (Django) exposes a REST API that powers features like tasks, focus mode, Bluetooth social interactions, XP and badges, and weekly leaderboards.

To choose the right production database, I first summarised the differences between SQL and NoSQL and then aligned them with the requirements of my app.

4.5.2.1 SQL Databases (Relational – e.g., PostgreSQL, MySQL)

SQL databases organise data in tables with rows and columns and enforce a predefined schema. "SQL databases organize records into tables with rows and columns," providing strong integrity and support for relationships(GeeksforGeeks, 2023).

Key characteristics I rely on:

- ACID transactions – important for XP updates, credit spending in the shop, and leaderboard ranking, where I must never double-award or lose points. [19]
- Strongly structured schema – suitable for clearly defined entities such as Users, Tasks, FocusSessions, SocialInteractions, Badges, and Leaderboard

entries.

- Powerful JOIN queries – needed to fetch combined data, for example “user with total XP + weekly XP + current league rank”.

4.5.2.2 NoSQL Databases (Non-relational – e.g., MongoDB, Firebase)

NoSQL databases store data in non-relational formats like key–value pairs or JSON documents and are optimised for schema flexibility and horizontal scaling. They are often used when relationships are loose and the schema changes frequently.

For example, a NoSQL representation of a social interaction in my app could look like:

```
{
  "interaction_id": "INT-12345",
  "user_a_id": "U001",
  "user_b_id": "U014",
  "timestamp": "2025-01-12T10:30:00Z",
  "reward_given": true,
  "context": {
    "location": "Library",
    "duration_minutes": 20
  }
}
```

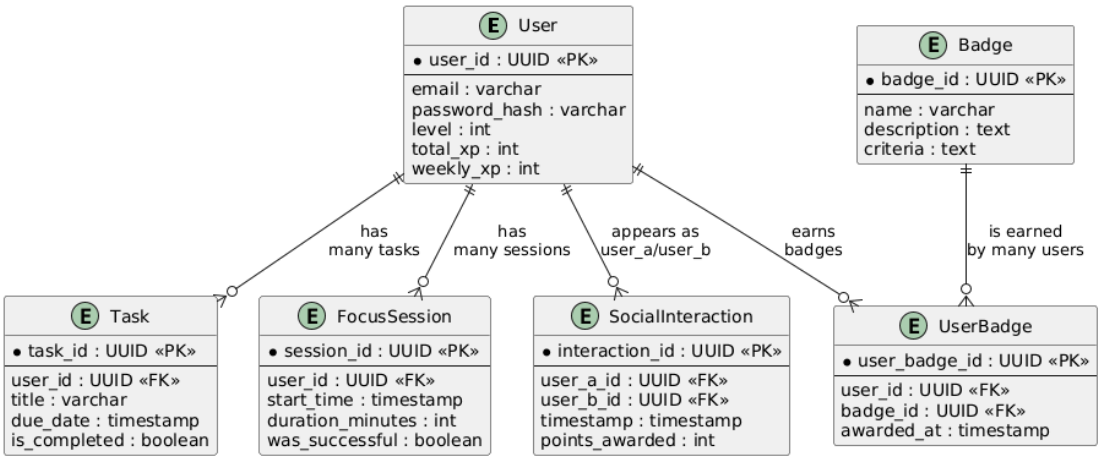
However, many core features of my app — such as accurate XP accounting, league-based leaderboards, and badge progression thresholds — require strong consistency and complex relational queries. These are areas where SQL databases are a better natural fit than NoSQL(Anderson and Nicholson, 2024).

4.5.3 ER Diagram for My Production Database

Based on the use cases in my Functional Specification (e.g. UC-AM-01 to UC-GM-06),

I designed the following core entities and relationships for my relational schema:

- User – accounts, XP, level, weekly points
- Task – personal tasks linked to a user
- FocusSession – timed focus sessions with result and duration
- SocialInteraction – Bluetooth-confirmed offline social interactions between two users
- Badge / UserBadge – catalogue of badges and a link table showing which badges each user has earned



4.5.4 Why I Choose PostgreSQL (Over MySQL and NoSQL) for Production

For the production environment of my app, I explicitly choose **PostgreSQL** as the main database engine.

1. **Better Concurrency Than SQLite**

   SQLite's file-locking approach means only one writer can modify the database file at a time; under heavy traffic, this easily becomes a bottleneck(Sqlite.org, 2025). PostgreSQL, by contrast, uses MVCC to allow readers and writers to work concurrently while still preserving ACID guarantees, which is essential when many students are completing tasks or focus sessions at the same time.

2. **Relational Model Fits My Gamified Domain**

   My core logic is strongly relational: each user can have many tasks, focus sessions, social interactions and earned badges; and leaderboard ranks depend on aggregated XP values. This aligns well with a SQL schema and JOIN queries, and less well with a schema-less NoSQL design.

3. **Advanced PostgreSQL Features**

   Compared with MySQL, PostgreSQL offers richer data-type support such as JSONB, arrays and range types, which I can use later for storing flexible metadata about interactions and timetables. Several evaluations show that PostgreSQL's JSONB implementation provides powerful indexing and query capabilities for semi-structured data.

4. **Long-Term Scalability and Integrity**

   As the user base grows from a small pilot to possibly an entire campus, PostgreSQL gives me a strong foundation for indexing, query optimisation, and partitioning without changing the application logic. This preserves the integrity of XP, credits, and social-interaction logs, even under higher load.

5. **Smooth Migration Path from SQLite**

   Because both SQLite and PostgreSQL are relational and well-supported by Django's ORM, I can start development quickly with SQLite and later run migrations to PostgreSQL with minimal changes in my application code(GeeksforGeeks, 2023).

In summary, I use **SQLite** in the early development phase to iterate quickly, and I adopt **PostgreSQL** in production to obtain the robustness, concurrency and data integrity that a real multi-user gamified student app requires.

4.6 API and Testing (DRF and Postman)

To enable communication between the back-end and front-end, I will use the Django REST framework (DRF) to create a simple REST API. At the same time, I will use tools like Postman to test the API during its construction. This way, I can ensure that the back-end logic can run independently before connecting it to the mobile application.

4.7 Version Control (Git and GitHub)

I will use Git and GitHub to save and manage all my code and documents. The specific operations will be carried out as needed.

4.8 Hosting

For the final demonstration of my project, I need to deploy my Django backend to a live server. During my research, I evaluated two main cloud service models: Infrastructure-as-a-Service (IaaS) and Platform-as-a-Service (PaaS). IaaS solutions—such as **Amazon Web Services (AWS)**(Amazon, 2019) or **Google Cloud**(Google, 2019)—provide maximum flexibility but require significant expertise in server administration, including operating system setup, web server configuration, database deployment, networking, and security hardening. Learning and configuring these components would introduce unnecessary overhead and distract me from focusing on the core features of my application.

Because of this, I decided that a PaaS solution is more suitable for this project.

Heroku

Heroku is a mature and widely adopted PaaS platform known for its simplicity and excellent support for Python/Django applications. Its official documentation states:

"Build Python apps and APIs on the Heroku platform… extend your app with 150+ add-ons(Heroku, 2025)."

Advantages of Heroku

- **Extremely easy deployment:** I can deploy my Django backend by pushing code to Heroku via Git, defining a simple Procfile, and configuring environment variables. Several guides note that Heroku is a “favorite cloud platform provider of many startups and developers” because of this simplicity(Bartosz Zaczyński, 2021).
- **Strong Django support:** Heroku integrates seamlessly with Django’s ecosystem, from database configuration to static file handling.
- **Rich ecosystem of add-ons:** Heroku provides built-in options for PostgreSQL, caching, logging, monitoring, and automated scaling.
- **Scalable architecture:** As my app scales, Heroku dynos can be upgraded without changing any backend architecture.

Disadvantages of Heroku

- **Free tier limitations:** Limited CPU and memory, and no background workers unless upgraded.
- **Higher cost at scale:** As the project evolves beyond a demonstration, costs can grow more quickly compared to Python-focused PaaS platforms.
- **Less infrastructure control:** Compared to IaaS, I cannot fully customise the underlying OS or networking settings(Heroku vs PythonAnywhere Comparison (2025) | Feature by Feature, 2025).

PythonAnywhere

PythonAnywhere is a specialized PaaS designed specifically for Python applications. It provides a browser-based environment to run, deploy, and manage Python web apps.

As described in a comparison article:

> “PythonAnywhere is a cloud-based Python development and hosting environment that requires no server installation and is ideal for smaller Django applications(StackShare, 2025).”

Advantages of PythonAnywhere

- **Beginner-friendly deployment:** Very simple setup for Django; ideal for lightweight prototypes.
- **Cost-effective:** Its pricing is often lower than Heroku for small-scale applications.
- **Python-focused environment:** Everything from virtual environments to log access is streamlined for Python developers.

Disadvantages of PythonAnywhere

- **Limited scaling:** Not designed for high-traffic or compute-heavy workloads; fewer scaling options than Heroku.
- **Smaller add-on ecosystem:** Fewer integrations and automation tools compared with Heroku.
- **Manual configuration required:** Steps like static file handling, domain settings, and virtual environment configuration require manual work.(Team, 2022)

5.Conclusion

In this project, I aimed to develop a gamified mobile application to help university students improve their study motivation, task organization, and real-world social interaction. As discussed in the introduction, the app focuses on transforming everyday activities into meaningful achievements through points, badges, and leaderboard systems, encouraging students to stay productive and engaged.

From the market and competitive analysis, I learned valuable lessons from existing applications such as Habitica, Forest, Headspace, and Duolingo. Each provided insights into effective gamification strategies and potential pitfalls. Based on this analysis, I decided to combine key motivational features—such as progress tracking, fair competition, and a calm interface—while simplifying complex game elements to maintain focus on student productivity and real-life interaction.

In the technology investigation, I selected a decoupled architecture using Django for the backend and Kotlin for the Android frontend to ensure scalability and reliability. The planned migration from SQLite to PostgreSQL will support multi-user performance, and deploying on Heroku allows for efficient testing and demonstration.

Overall, this research confirms both the demand and technical feasibility for a student-focused gamified app. It provides a clear foundation for further development and demonstrates how gamification principles can be effectively applied to support learning and personal growth in higher education.

Reference

Amazon (2019). *AWS Free Tier*. [online] Amazon Web Services, Inc. Available at: https://aws.amazon.com/free/.

anand (2010). *SQLite Concurrent Access*. [online] Stack Overflow. Available at: https://stackoverflow.com/questions/4060772/sqlite-concurrent-access.

Anderson, B. and Nicholson, B. (2024). *SQL vs. NoSQL Databases: What's the Difference? | IBM*. [online] Ibm.com. Available at: https://www.ibm.com/think/topics/sql-vs-nosql.

Bartosz Zaczyński (2021). *Hosting a Django Project on Heroku*. [online] Realpython.com. Available at: https://realpython.com/django-hosting-on-heroku [Accessed 17 Nov. 2025].

Cloke, H. (2021). *How to Boost Engagement in Your Learning Technology with Leaderboards*. [online] Growth Engineering. Available at: https://www.growthengineering.co.uk/gamification-leaderboards-lms/.

Delgado, C. (2025). *Android vs iOS: Which Mobile Platform Is Best For App Development In 2025?* [online] Our Code World. Available at: https://ourcodeworld.com/articles/read/2351/android-vs-ios-which-mobile-platform-is-best-for-app-development-in-2025.

Deterding, S., Dixon, D., Khaled, R. and Nacke, L. (2011). From Game Design Elements to gamefulness: Defining 'gamification'. *Proceedings of the 15th International Academic MindTrek Conference on Envisioning Future Media Environments - MindTrek '11*, 11, pp.9–15. doi:https://doi.org/10.1145/2181037.2181040.

Duolingo (2025). *Learn a Language for Free*. [online] Duolingo. Available at: https://www.duolingo.com/.

Forest (2016). *Forest*. [online] Forestapp.cc. Available at: https://www.forestapp.cc/.

GeeksforGeeks (2023). *SQL vs. NoSQL Which Database to Choose in System Design?* [online] GeeksforGeeks. Available at:

https://www.geeksforgeeks.org/system-design/which-database-to-choose-while-designing-a-system-sql-or-nosql [Accessed 17 Nov. 2025].

GeeksforGeeks (2024). *Monolithic Architecture System Design*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/system-design/monolithic-architecture-system-design/.

Google (2019). *Cloud Computing Services | Google Cloud*. [online] Google Cloud. Available at: https://cloud.google.com/.

Gupta, S. (2023). *Decoupled Architecture & Microservices - Saurabh Gupta - Medium*. [online] Medium. Available at: https://medium.com/@saurabh.engg.it/decoupled-architecture-microservices-29f7b201bd87.

Habitica (2020). *Habitica - Gamify Your Life*. [online] Habitica.com. Available at: https://habitica.com/.

Hamari, J. (2019). Gamification. *The Blackwell Encyclopedia of Sociology*, pp.1–3. doi:https://doi.org/10.1002/9781405165518.wbeos1321.

Harry (2022). *Gamification In Learning: How to Use Badges*. [online] Growth Engineering. Available at: https://www.growthengineering.co.uk/gamification-badges-lms/.

Headspace (2024). *Meditation and Mindfulness Made Simple - Headspace*. [online] Headspace. Available at: https://www.headspace.com/.

Heroku vs PythonAnywhere Comparison (2025) | Feature by Feature (2025). *Appmus - Find Software Alternatives*. [online] Appmus. Available at: https://appmus.com/vs/heroku-vs-pythonanywhere [Accessed 17 Nov. 2025].

Heroku. (2025). *Heroku*. [online] Available at: https://www.heroku.com/python [Accessed 17 Nov. 2025].

Jacob (2024). *Unleashing the Power of Points in Gamification: A Complete 2024 Guide! 🎮 - Gamification Hub*. [online] Gamification Hub. Available at:

https://www.gamificationhub.org/points-in-gamification/.

pythontutorials. (2025). *Django vs Flask: Which Python Framework Should You Choose?* [online] Available at: https://www.pythontutorials.net/django/django-vs-flask-which-python-framework-should-you-choose/ [Accessed 16 Nov. 2025].

Robson, K., Plangger, K., Kietzmann, J.H., McCarthy, I. and Pitt, L. (2015). Is It All a Game? Understanding the Principles of Gamification. *Business Horizons*, 58(4), pp.411–420. doi:https://doi.org/10.1016/j.bushor.2015.03.006.

Sqlite.org. (2025). *File Locking And Concurrency In SQLite Version 3*. [online] Available at: https://sqlite.org/lockingv3.html [Accessed 17 Nov. 2025].

StackShare (2025). *StackShare*. [online] Stackshare.io. Available at: https://stackshare.io/stackups/heroku-vs-pythonanywhere [Accessed 17 Nov. 2025].

Team, B.E. (2022). *Heroku vs PythonAnywhere - Which is the best option?* [online] Back4App Blog. Available at: https://blog.back4app.com/heroku-vs-pythonanywhere-which-is-the-best [Accessed 17 Nov. 2025].